Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică
Departamentul Ingineria Software și Automatică
Program: Software Engineering

Operating Systems

Laboratory Work No. 5

The File System with Your Own Hands

Track A: Warm-up on Linux

Done by: st. gr. FAF-241, Ermicev Sofia

Checked by: Patricia Reițman

Chișinău – 2026

Contents

1 Purpose of the Work **2**

2 Part 1. The Tree and File Types **2**

2.1 Observe answers 2

3 Part 2. Inodes and Links **3**

3.1 Observe answers 4

4 Part 3. Permissions **4**

4.1 Observe answers 5

5 Part 4. Disks and the Kernel as Files **5**

5.1 Observe answers 6

6 Conclusion **6**

1. Purpose of the Work

The purpose of this laboratory work is to explore Linux filesystem concepts through direct experiments: file types, inodes, hard and symbolic links, permissions, special permission bits, disk usage, block devices, and the `/proc` pseudo-filesystem.

2. Part 1. The Tree and File Types

```
$ ls -ld /etc /etc/passwd /bin /dev/null /tmp
lrwxrwxrwx  1 root root     7 Apr 22  2024 /bin -> usr/bin
crw-rw-rw-  1 root root  1, 3 Oct  9 02:09 /dev/null
drwxr-xr-x 98 root root  4096 Oct 12 15:40 /etc
-rw-r--r--  1 root root  1545 May  2  2025 /etc/passwd
drwxrwxrwt 13 root root 12288 Oct 12 15:35 /tmp
```

Figure 1.1 – Actual file types and permissions reported by ls -ld

```
$ du -sh /var/log 2>/dev/null
392M    /var/log
$ du -sh /usr 2>/dev/null
7.4G    /usr
```

Figure 1.2 – Actual disk-usage comparison of /var/log and /usr

2.1. Observe answers

1. The first character of `ls -l` identifies the object type: directory `d`, regular file `-`, symbolic link `l`, and character device `c`.
2. On this system `/bin` is a symbolic link to `usr/bin`.
3. `/dev/null` is a character device. `/dev/sdc` is a block device containing the root filesystem.
4. `/usr` was approximately 7.4 GB compared with 392 MB for `/var/log`, roughly nineteen times larger.

3. Part 2. Inodes and Links

```
$ stat notes.txt
File: notes.txt
Size: 6               Blocks: 8          IO Block: 4096   regular file
Device: 8,32    Inode: 44076       Links: 1
Access: (0644/-rw-r--r--)  Uid: (1000/sofia)  Gid: (1000/sofia)
```

Figure 2.1 – Actual inode metadata of notes.txt

```
$ ln notes.txt hard.txt
$ ls -li notes.txt hard.txt
44076 -rw-r--r-- 2 sofia sofia 6 Oct 12 15:42 hard.txt
44076 -rw-r--r-- 2 sofia sofia 6 Oct 12 15:42 notes.txt
$ rm notes.txt
$ cat hard.txt
hello
```

Figure 2.2 – Actual hard-link experiment

```
$ ln -s hard.txt soft.txt
$ ls -li hard.txt soft.txt
44076 -rw-r--r-- 1 sofia sofia 6 Oct 12 15:42 hard.txt
44078 lrwxrwxrwx 1 sofia sofia 8 Oct 12 15:43 soft.txt -> hard.txt
$ rm hard.txt
$ cat soft.txt
cat: soft.txt: No such file or directory
```

Figure 2.3 – Actual symbolic-link experiment

```
$ echo x > one.txt
$ ls -l one.txt
-rw-r--r-- 1 sofia sofia 2 Oct 12 15:43 one.txt
$ du -h one.txt
4.0K    one.txt
$ truncate -s 1G big.img
$ ls -lh big.img
-rw-r--r-- 1 sofia sofia 1.0G Oct 12 15:43 big.img
$ du -h big.img
0       big.img
```

Figure 2.4 – Actual logical-size and sparse-file experiment

3.1. Observe answers

1. `Inode: 44076` is the inode number; `Links: 1` is the number of hard-link directory entries referring to it.
2. Removing one hard-link name does not remove the data while another hard link still references the inode.
3. A symbolic link has its own inode and stores a pathname. After removing its target it becomes broken. Its size was 8 bytes because `hard.txt` has eight characters.
4. A new directory has link count 2; each subdirectory increases the parent's link count through its `..` entry.
5. Logical size and allocated space differ. The small file occupied one filesystem block, while the 1 GB sparse file initially occupied no data blocks.

4. Part 3. Permissions

```
$ echo secret > s.txt
$ chmod 600 s.txt
$ ls -l s.txt
-rw------- 1 sofia sofia 7 Oct 12 15:47 s.txt
$ chmod 000 s.txt
$ cat s.txt
cat: s.txt: Permission denied
$ chmod u+r s.txt
$ cat s.txt
secret
```

Figure 3.1 – Actual regular-file permission experiment

```
$ chmod u=r,go= box
$ ls box
ls: cannot access 'box/f': Permission denied
f
$ cat box/f
cat: box/f: Permission denied
$ chmod u=x,go= box
$ ls box
ls: cannot open directory 'box': Permission denied
```

```
$ cat box/f
hi
```

Figure 3.2 – Actual directory read-versus-execute permission experiment

4.1. Observe answers

1. Common modes include `600`, `644`, `755`, and `700`, combining owner/group/other read, write and execute permissions.
2. File ownership does not bypass ordinary permission bits; mode `000` prevented the normal owner process from reading the file.
3. With umask `0022`, new files derived from 666 became 644 and new directories derived from 777 became 755.
4. Directory read permission allows listing names, while execute permission allows pathname traversal and access to a known entry.
5. The setuid bit on `/usr/bin/passwd` allows controlled privileged updates, while the sticky bit on `/tmp` protects other users' files in the shared writable directory.

5. Part 4. Disks and the Kernel as Files

```
NAME MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
sda    8:0    0 388.4M  1 disk
sdb    8:16   0     4G  0 disk [SWAP]
sdc    8:32   0     1T  0 disk /mnt/wslg/distro
                               /
```

Figure 4.1 – Actual block devices reported by lsblk

```
$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/sdc       1007G   22G  934G   3% /
```

Figure 4.2 – Actual root-filesystem usage

```
$ grep -c processor /proc/cpuinfo
16
$ head -3 /proc/meminfo
MemTotal:       14186832 kB
MemFree:        13273344 kB
MemAvailable:   13354732 kB
```

Figure 4.3 – Actual CPU and memory information read through /proc

5.1. Observe answers

1. The WSL2 root filesystem is on `/dev/sdc`, mounted as ext4; `/dev/sdb` provides 4 GB of swap.
2. The root filesystem had a size of approximately 1007 GB, with 22 GB used and 934 GB available.
3. The environment exposed 16 logical processors and 14,186,832 kB of RAM (about 13.5 GiB).
4. `/proc/self` refers to the process that accesses it; therefore `head -5 /proc/self/status` reports information about `head` itself.

6. Conclusion

The experiments connected user-visible filesystem behavior with inodes, links, permission bits, allocated blocks, devices and process/kernel information. They also demonstrated the distinction between names and inodes, between logical and allocated size, and between directory read and execute permissions.